

Task Manager: 흠어진 Scheduler를 운영 실행 모델로 통합하기

Scheduler는 시간을 맞춰 실행하는 도구가 아니라, 실패와 재처리를 관리해야 하는 운영 실행 시스템이다.

요약

여러 모듈에 흠어진 scheduler와 batch 실행을 Task Manager로 통합했다. Manual, Cron, Queue task를 하나의 definition, run, retry, DLQ, audit 모델로 관리하고, 실행 결과와 parameter, actor, stage, tenant를 추적할 수 있도록 설계했다.

핵심 판단은 “배치 실행 도구”가 아니라 “운영 가능한 실행 모델”이 필요하다는 점이었다. 배치는 실패할 수 있고, 중복 실행될 수 있으며, 재처리와 보류, 취소, 수동 실행이 필요하다.

문제 정의

업무 시스템의 batch는 시간이 지나며 모듈별 scheduler, 임시 script, 수동 DB 작업, 외부 cron으로 흠어진다. 이 구조에서는 다음 문제가 반복된다.

- 어떤 batch가 언제 실행되었는지 확인하기 어렵다.
- 실패 원인과 parameter가 남지 않는다.
- 재처리 기준이 개발자 개인 판단에 의존한다.
- 중복 실행과 동시 실행을 통제하기 어렵다.
- 고객사별 실행 정책과 stage별 정책이 분리되지 않는다.
- queue 기반 비동기 실행과 cron 실행이 다른 방식으로 운영된다.

책임 경계

계층	책임
Task Definition	task type, schedule, params schema, timeout, concurrency 정책을 정의한다.
Scheduler	cron 기준으로 due task를 생성한다.
Queue	외부 요청이나 비동기 작업을 task run으로 전환한다.

계층	책임
Worker	DSL 또는 RPC 실행을 수행하고 결과를 기록한다.
DLQ / Retry	실패 작업을 재시도, 보류, 수동 복구 대상으로 분리한다.
Admin UI	실행 이력, 실패 원인, 재처리, 중지, 수동 실행을 제공한다.

핵심 설계 결정

Manual, Cron, Queue를 같은 실행 모델로 묶었다

Manual 실행, Cron 실행, Queue 실행은 시작 방식이 다르다. 그러나 실행 이후에는 동일한 질문을 가진다. 누가 실행했는가, 어떤 parameter였는가, 언제 시작했고 끝났는가, 실패했는가, 재처리 가능한가.

```
task_definition
-> task_run
  -> attempt
    -> result / error
      -> retry / dlq / completed
```

이 구조를 통해 실행 방식이 달라도 운영 화면과 audit 기준을 통일했다.

allowConcurrent와 lock을 명시했다

Batch에서 중복 실행은 치명적일 수 있다. 같은 고객사, 같은 기간, 같은 업무에 대해 두 run이 동시에 실행되면 데이터 정합성이 깨질 수 있다. 따라서 task definition에 concurrency 정책과 lock scope를 명시해야 했다.

동시 실행을 무조건 막는 것도 답은 아니다. 고객사별로 병렬 실행 가능한 task와 전체 단일 실행만 허용해야 하는 task를 구분해야 한다.

Params와 Result를 audit 대상으로 봤다

Batch 실패를 분석하려면 실행 parameter와 결과 요약이 남아야 한다. “실패했다”만으로는 재처리할 수 없다. 어떤 tenant, 어떤 기간, 어떤 조건으로 실행했는지, 몇 건을 처리했고 몇 건이 실패했는지 남겨야 한다.

DSL 실행과 RPC 실행을 모두 열어두었다

일부 task는 PlanVM DSL로 표현할 수 있고, 일부는 기존 service RPC나 worker logic을 호출해야 한다. Task Manager는 실행 방식을 하나로 강제하지 않고, 실행 계약과 운영 이력을 통일하는 방향으로 설계했다.

버린 선택과 이유

각 모듈이 자체 scheduler를 가지는 방식은 익숙하지만 운영 기준이 흩어진다. 실패 상태, 재시도, 실행 이력, manual trigger, 권한, 결과 확인 방식이 모듈마다 달라지면 운영자는 전체 batch 상태를 한 화면에서 설명할 수 없다.

Cron만 기준으로 보는 방식도 충분하지 않았다. 운영 자동화에는 정해진 시간에 실행되는 작업, 사용자가 직접 실행하는 작업, 외부 이벤트로 queue에 들어오는 작업이 모두 존재한다. 실행 진입점은 달라도 run state와 audit은 같은 모델로 관리되어야 했다.

실패 시 즉시 재시도하는 단순 구조도 피했다. 외부 시스템 장애, DB lock, 잘못된 파라미터, 데이터 정합성 오류는 서로 다른 재처리 기준을 가진다. 무조건 재시도는 장애를 키울 수 있다.

Task 운영에서 확인해야 하는 흔적

흔적	목적
definition version	어떤 정의로 실행되었는지 재현한다.
trigger source	cron, manual, queue, retry를 구분한다.
params snapshot	실행 당시 입력과 이후 변경을 분리한다.
run state transition	RUNNING, RETRY_WAIT, FAILED, SUCCESS 경로를 audit한다.
DLQ 사유	자동 재처리 불가 작업을 사람이 판단할 수 있게 한다.

상태 모델

Task 상태는 운영자가 이해할 수 있어야 한다. 단순 SUCCESS/FAIL 로는 부족하다.

- READY
- > ENQUEUED
 - > RUNNING
 - > COMPLETED
 - > FAILED
 - > RETRY_WAIT
 - > DLQ
 - > CANCELLED

각 상태는 전환 조건과 운영 액션을 가져야 한다. 예를 들어 ENQUEUED 상태가 일정 시간 이상 지속되면 worker 장애나 queue 지연으로 판단하고 별도 경고를 발생시켜야 한다.

운영 검증

검증 기준은 다음과 같았다.

- task 실행 이력이 tenant, stage, actor, params 기준으로 남는가.
- 실패한 task를 안전하게 재처리할 수 있는가.
- concurrency 정책이 실제 중복 실행을 막는가.
- 오래 대기 중인 task를 감지할 수 있는가.
- DLQ에 적재된 task를 운영자가 분석하고 복구할 수 있는가.
- scheduler, queue, worker 장애를 구분할 수 있는가.

역할과 책임

Task Manager의 task definition, run, attempt, retry, DLQ, concurrency, cron/queue/manual 통합 모델, Admin UI 운영 흐름, PlanVM/RPC 실행 연동 방향을 설계했다.

설계 결론

Batch는 background에서 조용히 실행되는 부가 기능이 아니다. 업무 데이터와 운영 상태를 바꾸는 실행 단위다. 따라서 batch는 API와 동일하게 권한, 입력 검증, 실행 이력, 실패 복구, audit 기준을 가져야 한다.